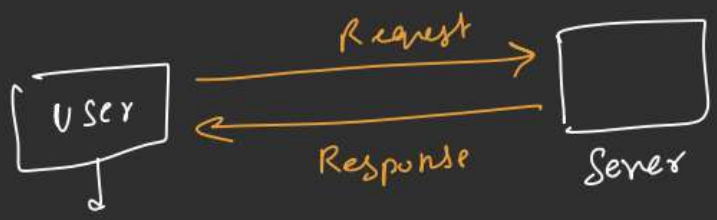
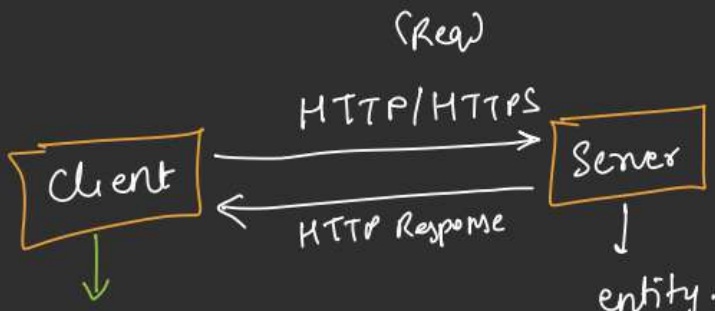
Spring Framework



Browser
(www.amazon.in)



Browser
Mopile App
React FE.
Android / IOS
Postman
Server

Client → Ask
Server → Respond

HTTP : Hyper Text Transfer Protocol.

- Request structure ?
- Response structure ?
- GET, POST, DELETE, PUT, PATCH.
- How data will be sent ?

HTTPS

HTTP

GET /courses.
HOST: www.coderarmy.in

Request:

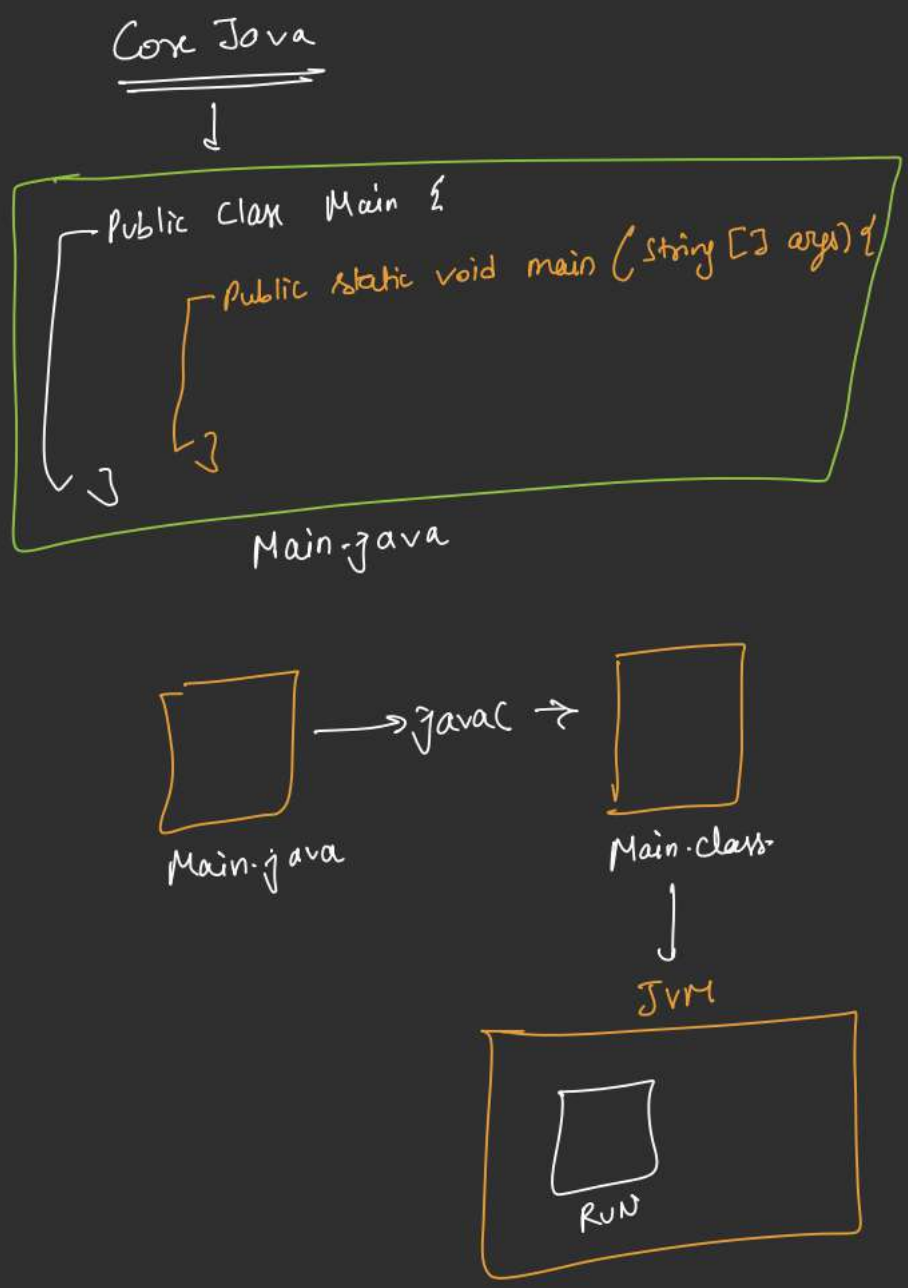
- Method Name
- URL / Path.
- Headers
- Body

```
POST /login
Host: www.coderarmy.in.
Content-Type: application/json
{
    email: ——
    password: ——
}
```

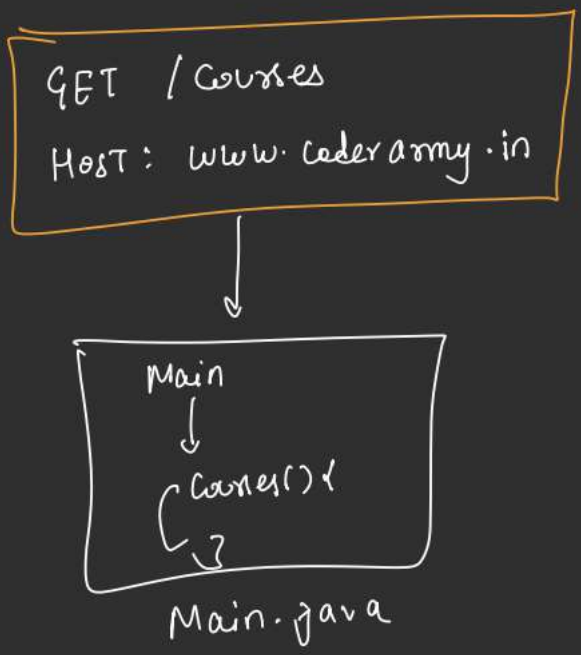
HTTP Response:

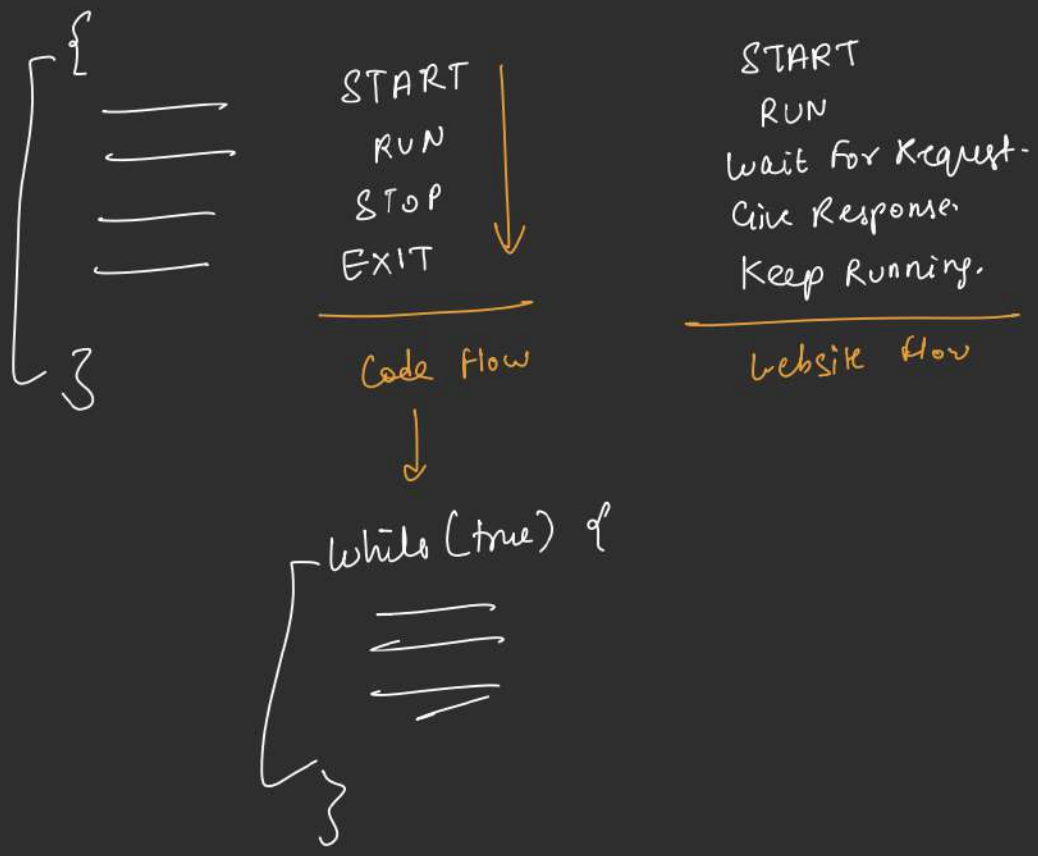
- status code (200 OK)
- Headers
- Body.

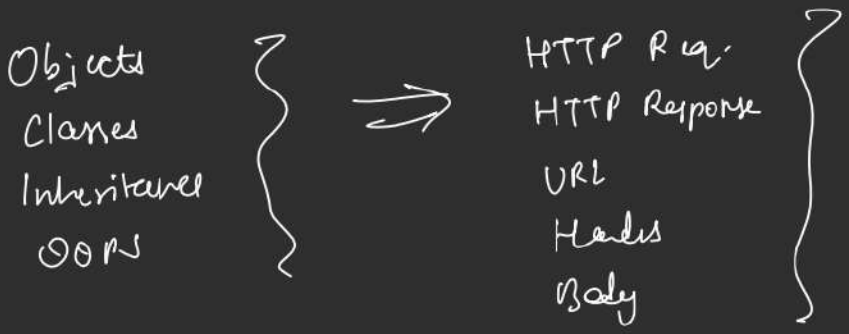
```
200 OK
ContentType: application/json.
{
message: "login successful"
}
```

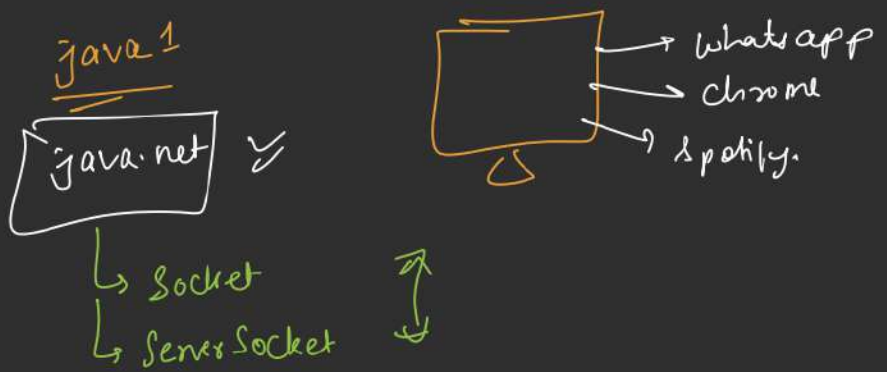

Core Java
Public class Main {
Public static void main (String [] args) {
}
}
Main.java

javac
Main.java
Main.class
JVM
RUN


GET /courses
Host: www.coderarmy.in
Main
courses() {
}
Main.java








```
ServerSocket server =
        new ServerSocket(8080);
```

GET /courses
Host: localhost:8080 (127.0.0.1)

↓

java.net → BR

1. Read I/P streams
2. Parse req manually.
3. Map a method to an end point.

```
if( endpoint.equals("courses") {
    //
}
else {
    //
}
```

4. Manually build HTTP Response.
5. Multi-Threading.

Servlet & Servlet Container → Server

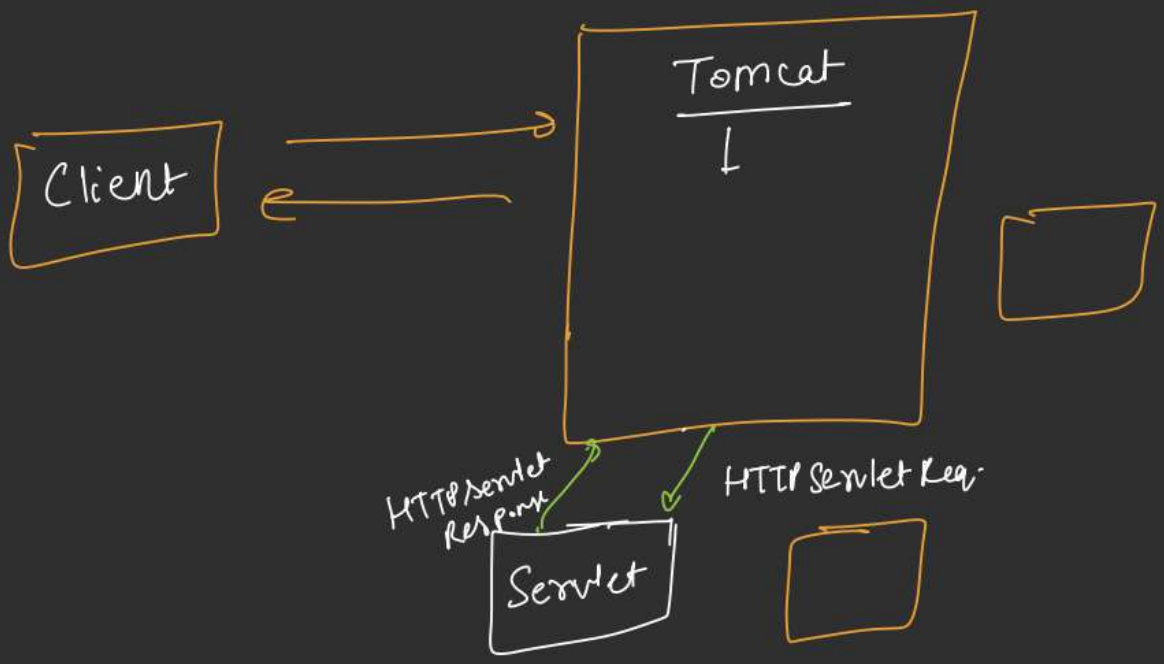
Tomcat
Jetty
undertow.

1997 Java EE

Servlet

Servlet Container (Tomcat)

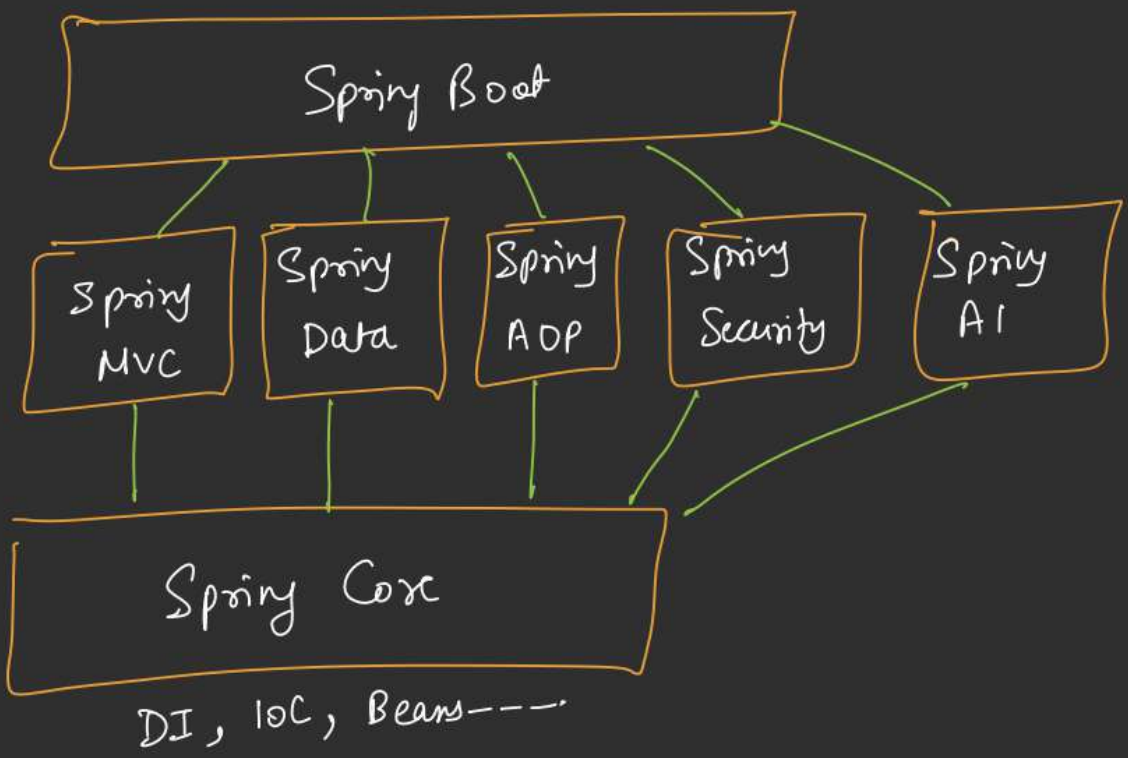
- Open port (say 8080)
- Read Incoming bytes
- Pass HTTP Req.
- Manage Threads
- Create HTTP Response



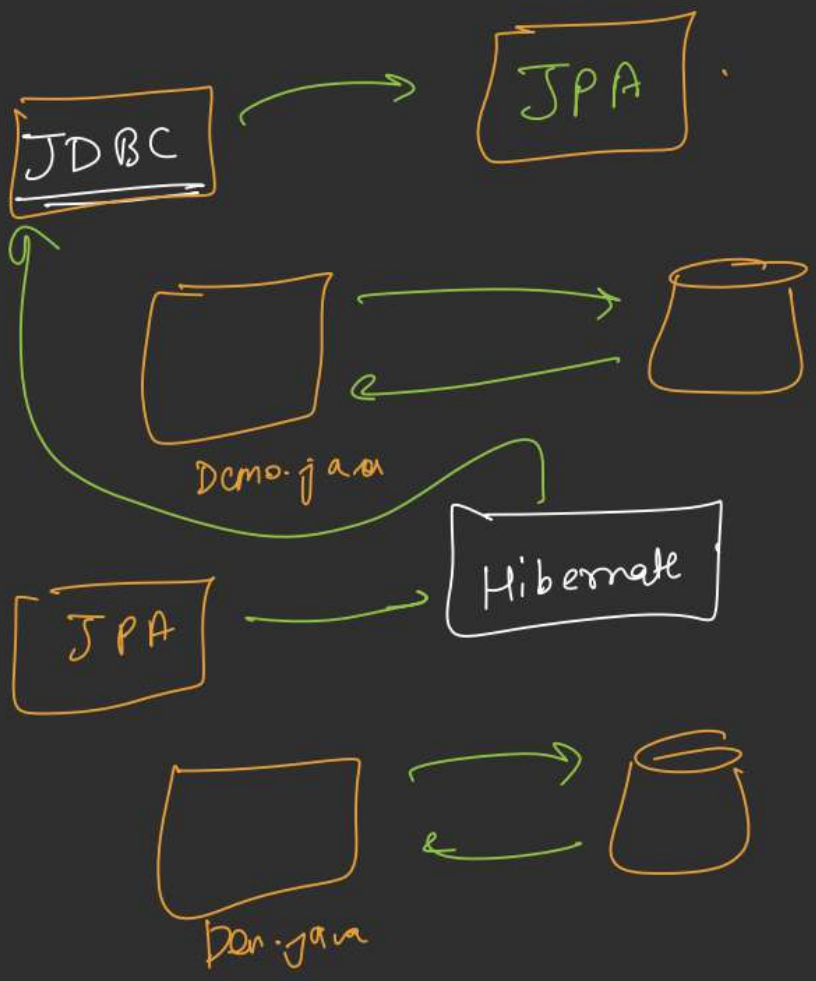
Spring Framework

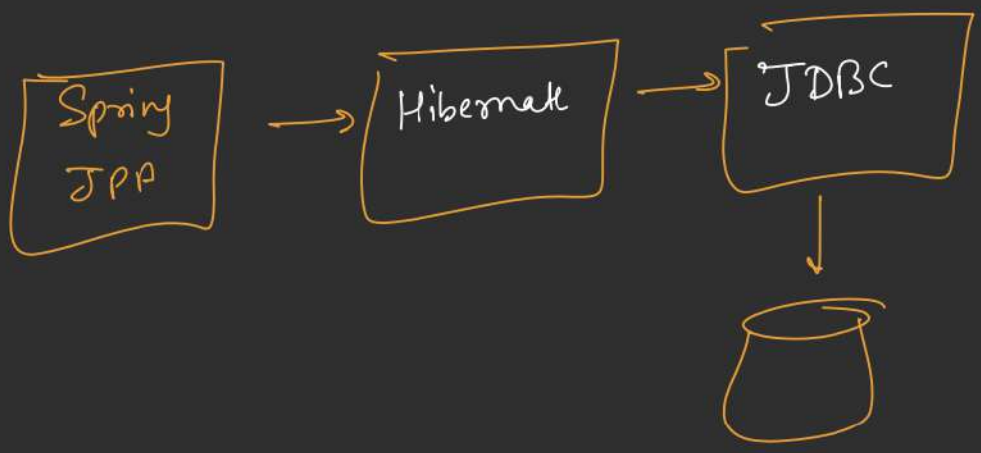
↓

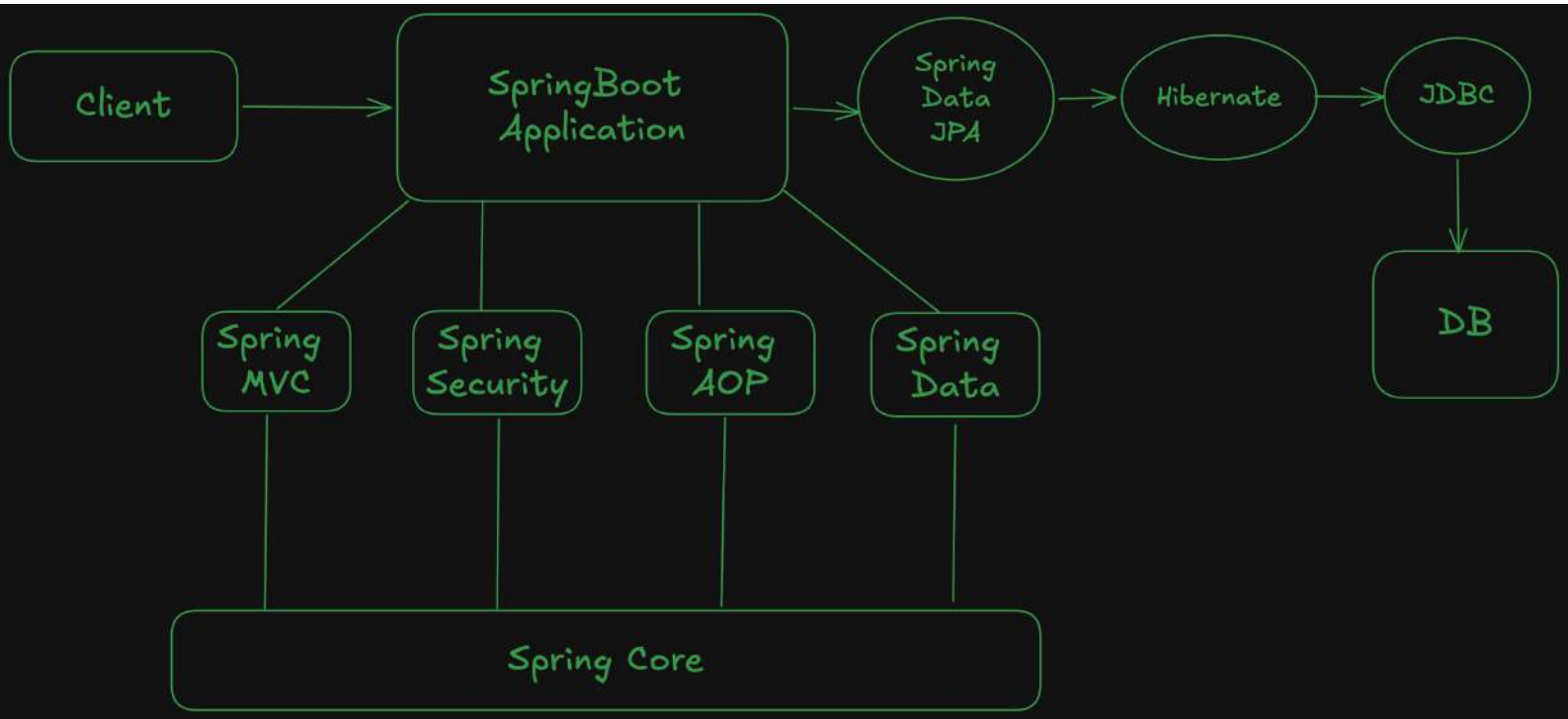
Ecosystem


Spring Boot
Spring MVC
Spring Data
Spring AOP
Spring Security
Spring AI
Spring Core
DI, IoC, Beans---


Spring Data


JDBC
JPA
Demo.java
JPA
Hibernate
Dao.java


Spring
JPA
Hibernate
JDBC



Java full stack developer